



Data and Artificial Intelligence

Cyber Shujaa Program

Week 9 Project Report
MLOPS: End-to-End Machine Learning Pipeline

Student Name: Amidu Dabor
Student ID: CS-DA01-25031
Date: 14 July, 2025

TABLE OF CONTENTS

1.0	INTRODUCTION	1
2.0	TASK COMPLETION	1
2.1	LOADING AND EXPLORING THE DATASET	1
2.2	DATA PREPARATION AND PREPROCESSING	2
2.3	MODEL IMPLEMENTATION AND HYPERPARAMETER TUNING	3
2.4	MODEL EVALUATION	5
2.5	MODEL DEPLOYMENT WITH FASTAPI.....	5
2.6	PUBLISHING AND SHARING NOTEBOOK.....	8
3.0	PORTFOLIO WEBSITE	8
4.0	CONCLUSION.....	8
	<i>Key Takeaways:</i>	8
5.0	REFERENCES.....	9

1.0 INTRODUCTION

This report presents the development of an end-to-end Machine Learning (ML) pipeline for predicting median house values in California using the California Housing Dataset provided by Scikit-learn. The objective was to build a regression model that follows MLOps best practices by incorporating:

- a. Data preprocessing
- b. Model selection and tuning
- c. Performance evaluation
- d. Deployment using a REST API

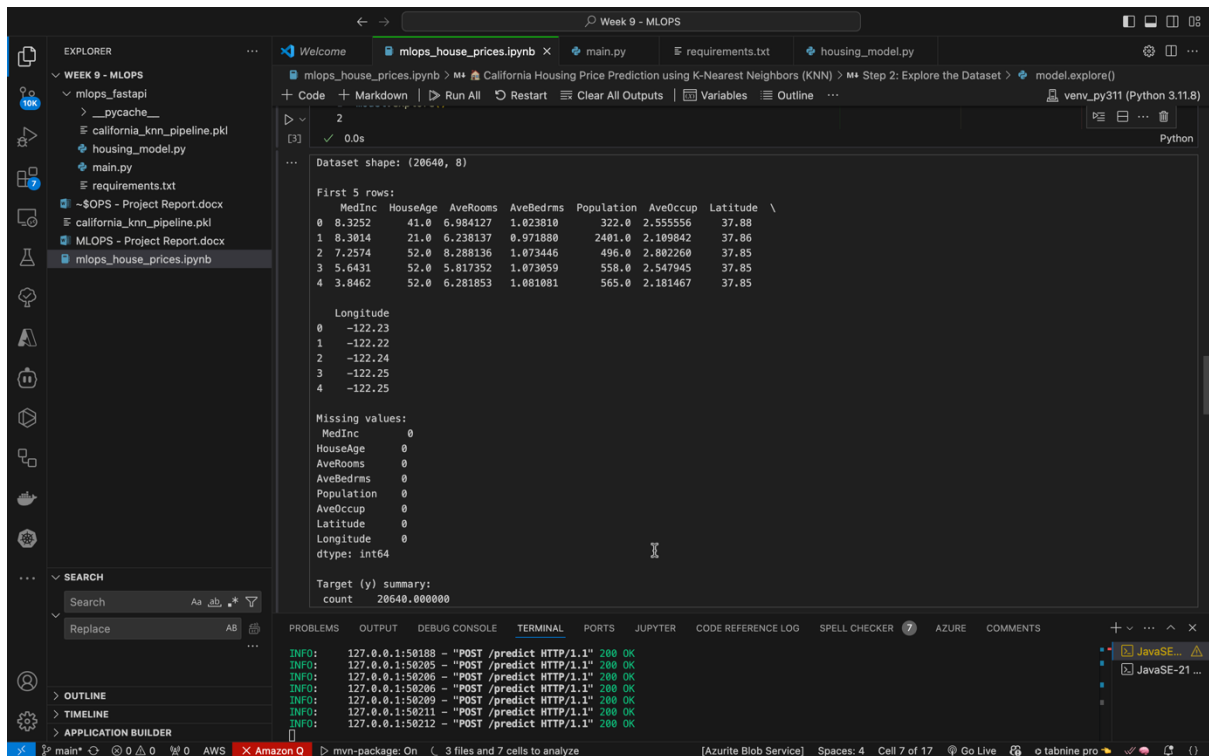
In addition to data engineering and modeling, this project emphasizes operationalization, deploying the trained model with FastAPI to serve real-time predictions. This hands-on experience mimics real-world workflows used in ML system development and deployment.

2.0 TASK COMPLETION

2.1 Loading and Exploring the Dataset

The dataset was loaded using the `fetch_california_housing()` function from Scikit-learn. The feature set includes 8 numerical features describing demographics and housing characteristics, while the target variable represents median house values.

Key exploration tasks included viewing the first few records of the dataset, checking for missing values, and examining summary statistics of both features and the target variable. All features were confirmed to be numeric, and no missing values were observed in the dataset.



```
Target (y) summary:
count    20640.000000
mean      2.068558
std       1.153956
min       0.149990
25%       1.196000
50%       1.797000
75%       2.647250
max       5.000010
Name: MedHouseVal, dtype: float64
```

2.2 Data Preparation and Preprocessing

Preprocessing was performed using a combination of Pipeline and ColumnTransformer. The entire dataset consisted of numeric values, so the following steps were applied to all features:

- Missing values (if any) were imputed using the mean strategy.
- Features were standardized using StandardScaler.

These transformations were packaged into a pipeline, which ensures that the same preprocessing steps will be applied during both training and inference. This is a key principle in MLOps to avoid data leakage and maintain consistency.

```
28     def prepare(self):
29         """Split and preprocess data using pipeline"""
30         self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(
31             self.X, self.y, test_size=0.2, random_state=42
32         )
33
34         numeric_features = self.X.columns.tolist()
35         numeric_transformer = Pipeline(steps=[
36             ('imputer', SimpleImputer(strategy='mean')),
37             ('scaler', StandardScaler())
38         ])
39
40         self.preprocessor = ColumnTransformer(transformers=[
41             ('num', numeric_transformer, numeric_features)
42         ])
43
44         self.pipeline = Pipeline(steps=[
45             ('preprocessor', self.preprocessor),
46             ('knn', KNeighborsRegressor())
47         ])
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
1  model.prepare()
2
[4] ✓ 0.0s
```

2.3 Model Implementation and Hyperparameter Tuning

The regression model used was KNeighborsRegressor, a non-parametric model that makes predictions based on the proximity of data points in feature space.

To improve the model's performance, a grid search with 5-fold cross-validation was applied to tune the following hyperparameters:

- a. `n_neighbors`: Number of neighbors to use (3, 5, 7, 9)
- b. `weights`: Uniform vs distance-based weighting
- c. `p`: Distance metric (1 = Manhattan, 2 = Euclidean)

The pipeline (preprocessing + model) was passed to GridSearchCV for tuning. The best model was selected based on the R^2 score.

```

49     def train_and_tune(self):
50         """Train using GridSearchCV with 5-fold cross-validation"""
51         param_grid = {
52             'knn__n_neighbors': [3, 5, 7, 9],
53             'knn__weights': ['uniform', 'distance'],
54             'knn__p': [1, 2]
55         }
56
57         self.grid_search = GridSearchCV(
58             estimator=self.pipeline,
59             param_grid=param_grid,
60             cv=5,
61             scoring='r2',
62             verbose=1,
63             n_jobs=-1
64         )
65
66         print("🔧 Training and tuning model...")
67         self.grid_search.fit(self.X_train, self.y_train)
68         self.best_model = self.grid_search.best_estimator_
69         print("Training complete!")
70
71         Tabnine | Edit | Test | Explain | Document
72     def evaluate(self):
73         """Evaluate the model"""
74         y_pred = self.best_model.predict(self.X_test)
75         r2 = r2_score(self.y_test, y_pred)
76         mse = mean_squared_error(self.y_test, y_pred)
77         # rmse = mean_squared_error(self.y_test, y_pred, squared=False)
78         rmse = np.sqrt(mse)
79
80         print("Best Params:", self.grid_search.best_params_)
81         print("CV R²:", round(self.grid_search.best_score_, 4))
82         print("Test R²:", round(r2, 4))
83         print("Test MSE:", round(mse, 4))
84         print("Test RMSE:", round(rmse, 4))

```

```

1  model.train_and_tune()
2
[5]  ✓ 5.0s

```

```

...  🔧 Training and tuning model...
    Fitting 5 folds for each of 16 candidates, totalling 80 fits
    Training complete!

```

```

1  model.evaluate()
2
[6]  ✓ 0.1s

```

```

...  Best Params: {'knn__n_neighbors': 9, 'knn__p': 1, 'knn__weights': 'distance'}
    CV R²: 0.7313
    Test R²: 0.7221
    Test MSE: 0.3642
    Test RMSE: 0.6034

```

2.4 Model Evaluation

The best model obtained from the grid search was evaluated on the held-out test set. The following performance metrics were computed:

- a. **R² Score:** Measures the proportion of variance explained by the model.
- b. **MSE (Mean Squared Error):** Average squared difference between predictions and actual values.
- c. **RMSE (Root Mean Squared Error):** The square root of MSE, easier to interpret.

The final R² score on the test set indicated good generalization capability of the model, and the RMSE was within a reasonable range.

```
[6] 1 model.evaluate()  
    2  
✓ 0.1s  
... Best Params: {'knn__n_neighbors': 9, 'knn__p': 1, 'knn__weights': 'distance'}  
    CV R²: 0.7313  
    Test R²: 0.7221  
    Test MSE: 0.3642  
    Test RMSE: 0.6034
```

The trained model was then saved as a .pkl file using Python's pickle module to be used later for deployment.

2.5 Model Deployment with FastAPI

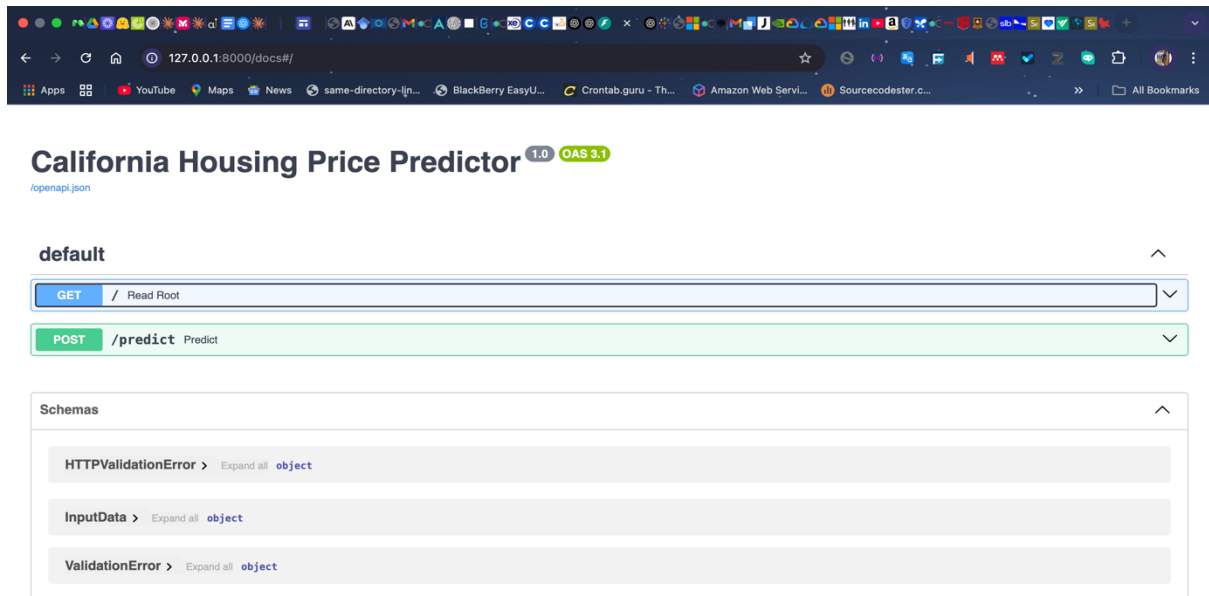
To simulate a production environment, a RESTful API was created using FastAPI. The API includes a /predict endpoint that accepts JSON input with the same structure as the training features and returns the predicted house value.

The API loads the saved model file and transforms incoming requests using the same pipeline. The following steps were performed:

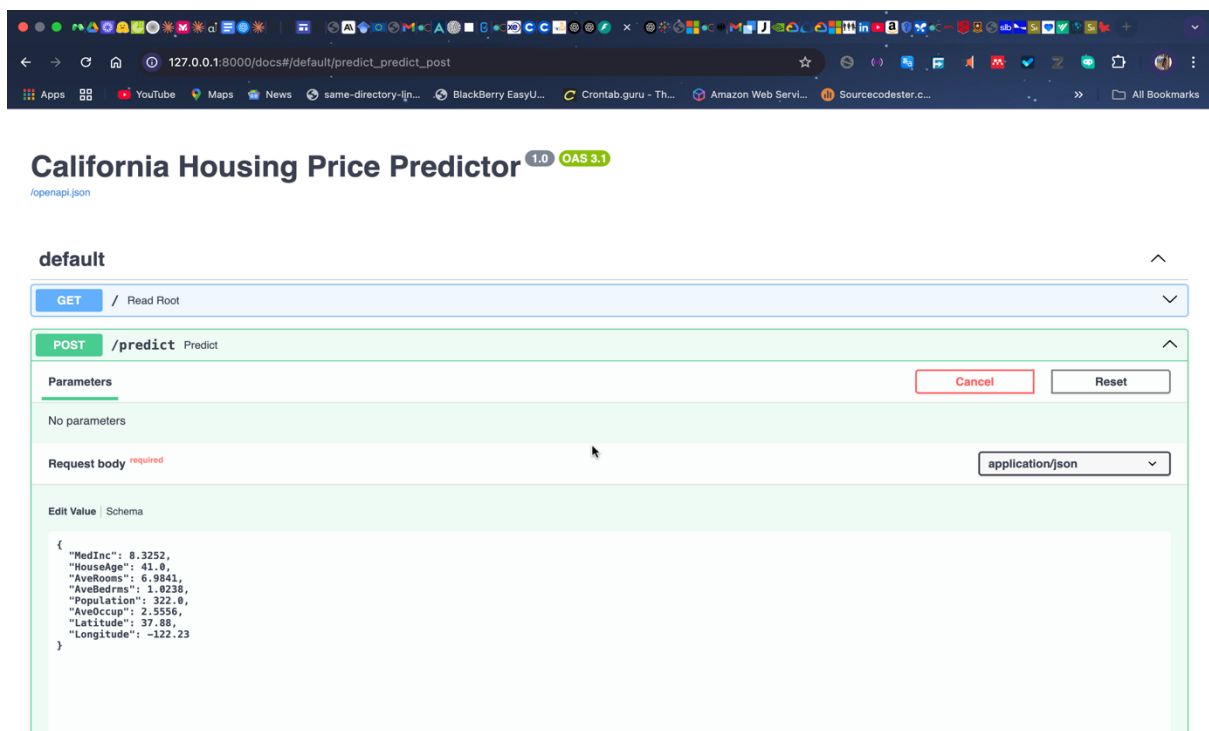
1. Defined a BaseModel schema using Pydantic.

2. Built the /predict route to parse input, run prediction, and return results.
3. Tested the API locally using curl, Postman, or the auto-generated Swagger UI.

Swagger UI



A sample prediction request (json)



Prediction response

The screenshot shows a web browser window with a REST client interface. The address bar shows the URL `127.0.0.1:8000/docs#/default/predict_predict_post`. The interface displays a POST request and its response.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "MedInc": 8.3252,
    "HouseAge": 41.0,
    "AveRooms": 6.9841,
    "AveBedres": 1.0238,
    "Population": 322.0,
    "AveOccup": 2.5556,
    "Latitude": 37.88,
    "Longitude": -122.23
  }'
```

Request URL

`http://127.0.0.1:8000/predict`

Server response

Code **Details**

200

Response body

```
{
  "predicted_house_value": 4.313
}
```

Response headers

```
content-length: 31
content-type: application/json
date: Mon, 14 Jul 2025 19:37:16 GMT
server: uvicorn
```

Responses

The screenshot shows a web browser window with a REST client interface. The address bar shows the URL `127.0.0.1:8000/docs#/default/predict_predict_post`. The interface displays a list of responses and a schema section.

Responses

Code	Description	Links
200	Successful Response	No links
422	Validation Error	No links

Media type

`application/json`

Example Value **Schema**

```
"string"
```

Media type

`application/json`

Example Value **Schema**

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

Schemas

This step finalizes the MLOps pipeline by making the trained model accessible through an API, enabling integration with web or mobile frontends or external systems.

2.6 Publishing and Sharing Notebook

The final notebook was uploaded to Google Drive (VS Code Notebook) and made publicly accessible.

🔗 VS Code Notebook Link: [<https://drive.google.com/drive/folders/1qAPSGXvND7K-DJT83AR3Xq-HrTrdg5h?usp=sharing>]

3.0 PORTFOLIO WEBSITE

Website Link: [<https://amidu-dabor.github.io/>]

4.0 CONCLUSION

This project provided a comprehensive walkthrough of the machine learning lifecycle, encompassing data loading, preprocessing, model training, hyperparameter tuning, evaluation, and deployment. By integrating these components into a cohesive workflow, we successfully applied foundational MLOps principles such as automation, reproducibility, and scalability.

The deployment of the trained model using FastAPI exemplified the transition from experimentation to production, enabling real-time predictions through a RESTful API. This step simulated a realistic operational environment where models are consumed by external applications or services.

Key Takeaways:

1. **Pipeline Design:** Using Pipeline and ColumnTransformer ensured that preprocessing steps were applied consistently during both training and inference, minimizing the risk of data leakage.
2. **Model Robustness:** Employing GridSearchCV with cross-validation enhanced model performance by systematically tuning hyperparameters while preventing overfitting.

3. **Deployment Readiness:** FastAPI offered an efficient, lightweight framework for deploying the model as a web service, emphasizing the importance of serving predictions in real-world ML systems.

In summary, this exercise deepened my understanding of not only supervised regression techniques but also the operational practices required to develop production-grade machine learning systems.

5.0 REFERENCES

1. **Scikit-learn Documentation**
<https://scikit-learn.org/stable/documentation.html>
2. **Pandas Documentation**
<https://pandas.pydata.org/docs/>
3. **NumPy Documentation**
<https://numpy.org/doc/>
4. **FastAPI Documentation:** <https://fastapi.tiangolo.com/>
5. **Python Official Documentation:** <https://docs.python.org/3/>
6. **Matplotlib Documentation:** <https://matplotlib.org/stable/contents.html>
7. **California Housing Dataset:** https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html